

CSC 226 Exercise 1
An Nguyen, V01073373
17 Jan 2026

Problem:

Consider the recurrence $T(n)$ below, which is defined for all integers $n \geq 0$.

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } n = 1 \\ T(n-1) + 2T(n-2) & \text{if } n \geq 2 \end{cases}$$

Prove that $T(n) \leq 2^n$ for all $n \geq 0$.

Proof:

Idea: We solve the recurrence relation using proof by **strong induction** on n .

Base Cases:

$$\begin{aligned} \text{For } n = 0 : \quad T(0) &= 1 = 2^0 = 1 \\ \text{For } n = 1 : \quad T(1) &= 1 < 2^1 = 2 \end{aligned}$$

Therefore, our base cases hold.

Induction Hypothesis: Suppose that $T(j) \leq 2^j$ for all $0 \leq j \leq k$, where $k \geq 1$.

Induction Step:

We must show that the statement holds for $n = k + 1$:

$$T(k+1) \leq 2^{k+1}$$

Using the recurrence relation, we have:

$$\begin{aligned} T(k+1) &= T(k) + 2T(k-1) \\ &\leq 2^k + 2 \cdot 2^{k-1} = 2^k + 2^k = 2 \cdot 2^k = 2^{k+1} \quad (\text{by I.H.}) \end{aligned}$$

Conclusion:

Therefore, by induction, the claim holds for all $n \geq 0$.

CSC 226 Exercise 2
An Nguyen, V01073373
24 Jan 2026

Problem:

We have seen that the Quickselect algorithm has worst-case $\theta(n)$ performance if pivots are selected with the BFPRT pivot selection scheme using sub-arrays of size 5. Prove that Quickselect has worst-case $\theta(n)$ performance if pivots are selected with the BFPRT pivot selection scheme using sub-arrays of size 7.

Proof:

Idea:

We'll compute the largest size of the two partitions created by using BFPRT algorithm with sub-arrays of size 7. Then, we'll set up the recurrence relation for Quickselect and solve it using induction on n .

Calculating the size of the partitions:

For an array of size n , we have $\lfloor \frac{n}{7} \rfloor$ sub-arrays of size 7 (ignore the remaining elements). After the partitioning is complete, each sub-array is sorted in $\theta(1)$ time since the size is constant. Since the sub-array size is odd, each sub-array will have a median after sorting.

We find the median of each sub-array, which gives us $\frac{n}{7}$ medians. The time taken to sort all sub-arrays and find their medians is $\theta(n)$. Suppose we arrange the sub-arrays in non-decreasing order based on their medians. Let p be the median of the middle sub-array. This value is the median of the sequence of medians, so we call it **median of medians**.

Since there are $\frac{n}{7}$ sub-arrays, p is greater than the $\lfloor \frac{n}{14} \rfloor$ medians and less than the $\lfloor \frac{n}{14} \rfloor$ medians. Each median is greater than 3 elements in its sub-array and less than 3 elements in its sub-array, so:

$$\begin{aligned} |L| &\geq \frac{n}{14} \times 4 = \frac{4n}{14} \\ |G| &\geq \frac{n}{14} \times 4 = \frac{4n}{14} \end{aligned}$$

(L is the set of elements $\leq p$, and G is the set of elements $\geq p$.)

Since $|L| + |G| + 1 = n$, the size of the largest partition is at most:

$$n - \frac{4n}{14} = \frac{10n}{14} = \frac{5n}{7}$$

Therefore, recursing on the largest partition takes at most $T\left(\frac{5n}{7}\right)$ time.

Computing the sequence of sub-array medians take $\theta(n)$ time, and finding p takes a recursive call to Quickselect on the sequence of $\frac{n}{7}$ medians, which takes $T\left(\frac{n}{7}\right)$ time.

Recurrence:

The overall running time of Quickselect with BFPRT pivoting can be modelled by the recurrence:

$$T(n) = \begin{cases} c_b & \text{if } n \leq 1 \\ T\left(\left\lfloor \frac{5n}{7} \right\rfloor\right) + T\left(\left\lfloor \frac{n}{7} \right\rfloor\right) + c_1n + c_2 & \text{if } n \geq 2 \end{cases}$$

Notice that $\frac{5n}{7} + \frac{n}{7} = \frac{6n}{7} < n$, so the size of the subproblems decreases geometrically. Let $c = c_b + c_1 + c_2$ and note that since $c_1n + c_2n$ for $n \geq 2$, the simplified recurrence below is always greater than $T(n)$:

$$T'(n) = \begin{cases} c & \text{if } n \leq 1 \\ T'\left(\left\lfloor \frac{5n}{7} \right\rfloor\right) + T'\left(\left\lfloor \frac{n}{7} \right\rfloor\right) + cn & \text{if } n \geq 2 \end{cases}$$

Claim: $T'(n) \leq 7cn$ for $n \geq 1$.

Proof: By induction on n .

Base Case: When $n = 1$, $T'(1) = c = cn \leq 7cn$. So the claim holds.

Induction Hypothesis: Suppose $T'(n) \leq 7cn$ for $n \in 1, 2, \dots, k$ for some $k \geq 1$.

Induction Step: Consider $T'(n+1)$:

$$\begin{aligned} T'(n+1) &= T'\left(\left\lfloor \frac{5(n+1)}{7} \right\rfloor\right) + T'\left(\left\lfloor \frac{n+1}{7} \right\rfloor\right) + c(n+1) \\ &\leq 7c \cdot \left(\left\lfloor \frac{5(n+1)}{7} \right\rfloor\right) + 7c \cdot \left(\left\lfloor \frac{n+1}{7} \right\rfloor\right) + c(n+1) \text{ (by I.H.)} \\ &\leq 7c \cdot \left(\frac{5(n+1)}{7}\right) + 7c \cdot \left(\frac{n+1}{7}\right) + c(n+1) \\ &\leq 5c(n+1) + c(n+1) + c(n+1) \\ &= 7cn \end{aligned}$$

Therefore, the claim holds for $n+1$ and, by induction, the claim holds for all $n \geq 1$.